Comprehensive Unit Testing Report

Automated Unit Test Suites Execution, Functional Verification & Isolation Quality Assurance

Project: SE Mini Project (SE_MP)	**Report Type:** Automated Unit Test Execution Report
Target Branch: `feature/ui-portal-dashboard`	**Pull Request:** #72 (Merged to `main`)
Environment: Node.js v22.x | PostgreSQL 16 | Redis 7	**Execution Engine:** Jest v29.7.0 (In-Band Runner)
Generated: November 06, 2026 - 14:34:29 IST	**Verification Result:** **ALL CRITERIA SATISFIED (0 ERRORS)**

1. Executive Summary & Quality Gates

This document presents the official verification report for the isolated unit testing phase of the **SE Mini Project (SE_MP)**. Unit testing evaluates domain logic, cryptographic functions, access controls, message ordering, and presence caching without external infrastructure dependencies. All unit test suites executed with **100% pass rate (0 failures, 0 regressions)**.

114	114	0	8	100.0%	24.00s
TOTAL TESTS	PASSED (100%)	FAILED	TEST SUITES	PASS RATE	EXECUTION TIME

2. Unit Test Suites Summary

Test Suite File	Tests	Passed	Failed	Duration	Status
`tests/auth.test.js`	21	21	0	0.00s	PASSED
`tests/branchCoverage.test.js`	56	56	0	0.00s	PASSED
`tests/registration.test.js`	7	7	0	0.00s	PASSED
`tests/services.test.js`	23	23	0	0.00s	PASSED
`tests/pushNotification.test.js`	3	3	0	0.00s	PASSED
`tests/roomModel.test.js`	2	2	0	0.00s	PASSED
`tests/offlineMessageService.test.js`	1	1	0	0.00s	PASSED
`tests/messageModel.pagination.test.js`	1	1	0	0.00s	PASSED

3. Detailed Test Case Catalog

The following catalog provides the comprehensive record of all 114 unit test assertions across the 8 test suites:

#	Suite / Domain	Test Case Description	Duration	Verdict
1	`auth.test.js`	Security Requirement: Password Hashing with Salt (Issue #7 - CHAT-SR-002, TC-Sec-02) **TC-Sec-02.1: should hash passwords using bcrypt with standard salt rounds (>= 10)**	210ms	PASS

#	Suite / Domain	Test Case Description	Duration	Verdict
2	auth.test.js	Security Requirement: Password Hashing with Salt (Issue #7 - CHAT-SR-002, TC-Sec-02) **TC-Sec-02.2: should generate distinct salt/hash even for identical passwords**	826ms	PASS
3	auth.test.js	Security Requirement: Password Hashing with Salt (Issue #7 - CHAT-SR-002, TC-Sec-02) **TC-Sec-02.3: verifyPassword() should correctly authenticate valid and reject invalid passwords**	608ms	PASS
4	auth.test.js	Security Requirement: Password Hashing with Salt (Issue #7 - CHAT-SR-002, TC-Sec-02) **TC-Sec-02.4: hashPassword() should reject invalid or empty password inputs**	10ms	PASS
5	auth.test.js	Security Requirement: Password Hashing with Salt (Issue #7 - CHAT-SR-002, TC-Sec-02) **TC-Sec-02.5: UserModel toJSON() should never expose password hash or plaintext password**	1ms	PASS
6	auth.test.js	Security Requirement: Password Hashing with Salt (Issue #7 - CHAT-SR-002, TC-Sec-02) **TC-Sec-02.6: registerUser() should store hashed password rather than plaintext**	710ms	PASS
7	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) **TC-Auth-02.1: generateToken() creates a valid verifiable JWT with expected user claims**	12ms	PASS
8	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) **TC-Auth-02.2: loginUser() successfully authenticates via email and password**	379ms	PASS
9	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) **TC-Auth-02.3: loginUser() successfully authenticates via username and password**	422ms	PASS
10	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) **TC-Auth-02.4: loginUser() rejects incorrect password with 401 Invalid credentials**	421ms	PASS
11	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) **TC-Auth-02.5: loginUser() rejects non-existent user with 401 Invalid credentials**	1ms	PASS
12	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) **TC-Auth-02.6: loginUser() rejects missing input credentials with 400**	1ms	PASS
13	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) › REST API POST /api/auth/login integration **TC-Auth-02.7: POST /api/auth/login returns 200 and JWT on correct credentials**	448ms	PASS
14	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) › REST API POST /api/auth/login integration **TC-Auth-02.8: POST /api/auth/login returns 401 on incorrect credentials**	424ms	PASS
15	auth.test.js	Functional Requirement: User Authentication & JWT Session Token (Issue #12 - CHAT-F-002, TC-Auth-02) › REST API POST /api/auth/login integration **TC-Auth-02.9: POST /api/auth/login returns 400 on missing credentials**	11ms	PASS
16	auth.test.js	Functional Requirement: Account Lockout after Repeated Failed Logins (Issue #13 - CHAT-F-003, TC-Auth-03) **TC-Auth-03.1: first 5 consecutive failed attempts return 401 Invalid credentials**	2124ms	PASS

#	Suite / Domain	Test Case Description	Duration	Verdict
17	auth.test.js	Functional Requirement: Account Lockout after Repeated Failed Logins (Issue #13 - CHAT-F-003, TC-Auth-03) **TC-Auth-03.2: 6th consecutive failure returns 429 Account Locked**	2106ms	PASS
18	auth.test.js	Functional Requirement: Account Lockout after Repeated Failed Logins (Issue #13 - CHAT-F-003, TC-Auth-03) **TC-Auth-03.3: records audit log entry when account lockout occurs**	2130ms	PASS
19	auth.test.js	Functional Requirement: Account Lockout after Repeated Failed Logins (Issue #13 - CHAT-F-003, TC-Auth-03) **TC-Auth-03.4: successful login resets the failed attempts counter**	1782ms	PASS
20	auth.test.js	Functional Requirement: Account Lockout after Repeated Failed Logins (Issue #13 - CHAT-F-003, TC-Auth-03) **TC-Auth-03.5: lockout automatically expires after 5 minutes**	2385ms	PASS
21	auth.test.js	Functional Requirement: Account Lockout after Repeated Failed Logins (Issue #13 - CHAT-F-003, TC-Auth-03) **TC-Auth-03.6: REST API POST /api/auth/login returns 429 on 6th failed attempt**	2113ms	PASS
22	branchCoverage.test.js	Sanitizer Utilities Branch Coverage **handles non-string values gracefully**	4ms	PASS
23	branchCoverage.test.js	Sanitizer Utilities Branch Coverage **escapes all special HTML characters**	13ms	PASS
24	branchCoverage.test.js	Sanitizer Utilities Branch Coverage **detects unsafe and safe markup**	3ms	PASS
25	branchCoverage.test.js	Sanitizer Utilities Branch Coverage **deeply sanitizes complex objects, arrays, and primitive values**	6ms	PASS
26	branchCoverage.test.js	Middlewares Branch Coverage › TLS Middleware **redirects GET and HEAD on insecure connection when enforced**	9ms	PASS
27	branchCoverage.test.js	Middlewares Branch Coverage › TLS Middleware **passes through when TLS enforcement is disabled and connection is not secure**	1ms	PASS
28	branchCoverage.test.js	Middlewares Branch Coverage › Error Middleware **handles not found route**	2ms	PASS
29	branchCoverage.test.js	Middlewares Branch Coverage › Error Middleware **handles entity.too.large error**	2ms	PASS
30	branchCoverage.test.js	Middlewares Branch Coverage › Error Middleware **handles generic server errors with defaults**	1ms	PASS
31	branchCoverage.test.js	Middlewares Branch Coverage › Error Middleware **handles custom status and code without message**	1ms	PASS
32	branchCoverage.test.js	Middlewares Branch Coverage › XSS Middleware **preserves password and deviceToken while sanitizing rest of body**	2ms	PASS
33	branchCoverage.test.js	Middlewares Branch Coverage › XSS Middleware **handles requests without body, query, or params**	1ms	PASS
34	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **rejects unauthenticated requests**	1ms	PASS
35	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **bypasses room check for platform administrator**	1ms	PASS
36	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **rejects requests with missing roomId**	2ms	PASS
37	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **accepts x-room-role header override**	14ms	PASS

#	Suite / Domain	Test Case Description	Duration	Verdict
38	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **uses mock room roles correctly**	1ms	PASS
39	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **rejects when member role does not match allowed roles**	1ms	PASS
40	branchCoverage.test.js	Middlewares Branch Coverage › RBAC Middleware **requirePlatformAdmin checks user role correctly**	1ms	PASS
41	branchCoverage.test.js	Audit Service Branch Coverage **validates required audit parameters**	33ms	PASS
42	branchCoverage.test.js	Audit Service Branch Coverage **redacts sensitive fields in metadata including nested objects and arrays**	19ms	PASS
43	branchCoverage.test.js	Audit Service Branch Coverage **filters in-memory audit logs by roomId, actorId, and date**	17ms	PASS
44	branchCoverage.test.js	Moderation Service & Controller Branch Coverage **validates required parameters for all moderation actions**	7ms	PASS
45	branchCoverage.test.js	Moderation Service & Controller Branch Coverage **protects room owner from being muted, banned, or removed**	1ms	PASS
46	branchCoverage.test.js	Moderation Service & Controller Branch Coverage **successfully executes mute, unmute, ban, unban, deleteMessage, removeMember**	97ms	PASS
47	branchCoverage.test.js	Moderation Service & Controller Branch Coverage **exercises ModerationController routes and error handlers**	51ms	PASS
48	branchCoverage.test.js	Media Service & Controller Branch Coverage **validates file upload format, size, and extension**	6ms	PASS
49	branchCoverage.test.js	Media Service & Controller Branch Coverage **validates room membership before upload**	1ms	PASS
50	branchCoverage.test.js	Media Service & Controller Branch Coverage **exercises media controller error branches**	1ms	PASS
51	branchCoverage.test.js	Controllers Branch Coverage (Room, Auth) **roomController validates required fields**	1ms	PASS
52	branchCoverage.test.js	Controllers Branch Coverage (Room, Auth) **authController validates missing credentials**	1ms	PASS
53	branchCoverage.test.js	Socket Handlers Branch Coverage **messageHandler handles invalid payload branches**	2ms	PASS
54	branchCoverage.test.js	Socket Handlers Branch Coverage **presenceHandler handles typing and session events**	4ms	PASS
55	branchCoverage.test.js	Database Models Branch Coverage **constructs models with appropriate defaults and toJSON**	1ms	PASS
56	branchCoverage.test.js	Database Models Branch Coverage **exercises model query branch edge cases with mocked db queries**	4ms	PASS
57	branchCoverage.test.js	Lockout Service Branch Coverage **handles null and undefined identifiers safely**	1ms	PASS
58	branchCoverage.test.js	Lockout Service Branch Coverage **increments failed attempts and triggers lockout at 5 failures**	13ms	PASS
59	branchCoverage.test.js	Object Storage Service Branch Coverage **validates missing configuration and throws appropriate errors**	7ms	PASS
60	branchCoverage.test.js	Object Storage Service Branch Coverage **exercises sendRequest with mocked HTTP transport**	6ms	PASS

#	Suite / Domain	Test Case Description	Duration	Verdict
61	branchCoverage.test.js	Notification Controller Branch Coverage **handles device registration and unregistration success and error flows**	2ms	PASS
62	branchCoverage.test.js	Redis Client & Presence Cache Branch Coverage **exercises redis client getters and health check branches**	4ms	PASS
63	branchCoverage.test.js	Message History Service Branch Coverage **validates required parameters and membership**	6ms	PASS
64	branchCoverage.test.js	Message History Service Branch Coverage **validates limit and cursor boundaries**	1ms	PASS
65	branchCoverage.test.js	Message History Service Branch Coverage **correctly handles pagination and nextCursor when hasMore is true or false**	1ms	PASS
66	branchCoverage.test.js	Auth Middleware Branch Coverage **rejects missing or malformed Authorization headers**	2ms	PASS
67	branchCoverage.test.js	Auth Middleware Branch Coverage **authenticates valid token and attaches user to request**	3ms	PASS
68	branchCoverage.test.js	Socket Handlers Delivery Branch Coverage **delivers direct message to online recipient and offline recipient**	14ms	PASS
69	branchCoverage.test.js	Socket Handlers Delivery Branch Coverage **presenceHandler processes room:join, room:leave, and typing events**	5ms	PASS
70	branchCoverage.test.js	Redis Pub/Sub Branch Coverage **validates event handler function argument**	7ms	PASS
71	branchCoverage.test.js	Additional Branch Boosters for Presence, Messaging, and Redis **presenceService exercises session lifecycle, parsing fallbacks, and user status**	4ms	PASS
72	branchCoverage.test.js	Additional Branch Boosters for Presence, Messaging, and Redis **messagingService fallback branches when database throws**	1ms	PASS
73	branchCoverage.test.js	Additional Branch Boosters for Presence, Messaging, and Redis **redisClient reconnectStrategy delay calculations and MAX_RETRIES**	5ms	PASS
74	branchCoverage.test.js	Additional Branch Boosters for Presence, Messaging, and Redis **redisPubSub publishMessage, publishRoomEvent, publishPresenceEvent when publisher is ready**	1ms	PASS
75	branchCoverage.test.js	Additional Branch Boosters for Presence, Messaging, and Redis **loggerMiddleware finish callback with url fallback and non-test log**	1ms	PASS
76	branchCoverage.test.js	Socket Gateway Branch Coverage **exercises socket handshake auth and TLS middleware branches**	7ms	PASS
77	branchCoverage.test.js	Socket Gateway Branch Coverage **exercises handleRedisEvent for MESSAGES, ROOM_EVENTS, and PRESENCE**	4ms	PASS
78	registration.test.js	Issue #11: User Registration **registers a valid user and stores a hashed password**	839ms	PASS
79	registration.test.js	Issue #11: User Registration **rejects an existing email with HTTP 409**	1ms	PASS
80	registration.test.js	Issue #11: User Registration **rejects an existing username with HTTP 409**	1ms	PASS
81	registration.test.js	Issue #11: User Registration **normalizes email addresses to lowercase and trims usernames**	414ms	PASS
82	registration.test.js	Issue #11: User Registration **rejects missing or invalid input with HTTP 400**	1ms	PASS

#	Suite / Domain	Test Case Description	Duration	Verdict
83	registration.test.js	Issue #11: User Registration **rejects a PostgreSQL unique-constraint race with HTTP 409**	503ms	PASS
84	registration.test.js	Issue #11: User Registration **propagates unexpected database errors instead of creating a fake user**	1ms	PASS
85	services.test.js	Auth service **returns registered user details without exposing the password**	390ms	PASS
86	services.test.js	Auth service **returns a login token and matching user email**	2ms	PASS
87	services.test.js	Auth service **caps configured JWT lifetimes at 24 hours**	1ms	PASS
88	services.test.js	Room service **creates a room with its owner as the initial member**	2ms	PASS
89	services.test.js	Room service **rejects a blank room name or missing owner**	1ms	PASS
90	services.test.js	Room service **joins a public room and returns its members**	2ms	PASS
91	services.test.js	Room service **rejects joining a nonexistent room**	1ms	PASS
92	services.test.js	Room service **rejects joining without an authenticated user**	1ms	PASS
93	services.test.js	Room service **rejects joining a private room without an invitation**	1ms	PASS
94	services.test.js	Room service **allows joining a private room with a valid invitation**	1ms	PASS
95	services.test.js	Room service **rejects an invalid private-room invitation**	1ms	PASS
96	services.test.js	Room service **creates a private-room invitation for the owner**	1ms	PASS
97	services.test.js	Room service **rejects invitation creation by an ordinary room member**	1ms	PASS
98	services.test.js	Room service **rejects invitation creation for a public room**	1ms	PASS
99	services.test.js	Room service **rejects invitation expiry greater than seven days**	1ms	PASS
100	services.test.js	Room service **removes a member when requested by the room owner (CHAT-F-023, TC-Room-04)**	2ms	PASS
101	services.test.js	Room service **prohibits non-owners from removing room members (AC-CHAT-F-023)**	1ms	PASS
102	services.test.js	Room service **prevents removing the room owner**	1ms	PASS
103	services.test.js	Messaging service **trims message text and preserves correlation metadata**	1ms	PASS
104	services.test.js	Messaging service **rejects empty or whitespace-only message text**	1ms	PASS

#	Suite / Domain	Test Case Description	Duration	Verdict
105	`services.test.js`	Messaging service **persists sent room messages to PostgreSQL database (CHAT-F-040, AC-CHAT-F-040)**	1ms	**PASS**
106	`services.test.js`	Messaging service **marks delivered messages in PostgreSQL database (CHAT-F-040)**	1ms	**PASS**
107	`services.test.js`	Messaging service **retrieves persisted message by ID (CHAT-F-040)**	1ms	**PASS**
108	`pushNotification.test.js`	FCM push notification service (CHAT-F-061, TC-Notif-02) **registers and removes a user-owned FCM device token**	3ms	**PASS**
109	`pushNotification.test.js`	FCM push notification service (CHAT-F-061, TC-Notif-02) **dispatches the direct message to registered devices using FCM HTTP v1**	262ms	**PASS**
110	`pushNotification.test.js`	FCM push notification service (CHAT-F-061, TC-Notif-02) **does not request FCM credentials when the recipient has no registered device**	1ms	**PASS**
111	`roomModel.test.js`	RoomModel.create **persists the room and owner membership atomically**	3ms	**PASS**
112	`roomModel.test.js`	RoomModel.create **rolls back room creation if owner membership insert fails**	8ms	**PASS**
113	`offlineMessageService.test.js`	Offline message queue **drains queued messages in FIFO order and clears the recipient queue**	2ms	**PASS**
114	`messageModel.pagination.test.js`	MessageModel.findByRoomId pagination **uses a composite cursor and returns tied-timestamp messages chronologically**	5ms	**PASS**

4. Architectural Isolation & Mocking Strategy

Isolated Execution Architecture: Unit tests are fully decoupled from real TCP ports and external dependencies. PostgreSQL operations are mocked via synchronous promise resolutions with query pattern matching; Redis pub/sub operations use an in-memory Node.js `EventEmitter` bus; and AWS S3 object storage uses a spied HTTP client transport. This guarantees high execution speed (< 25s for 114 tests) and 100% deterministic reproducibility across local development machines and CI runners.